

PW7

November 25, 2025

```
[1]: #Die ganzen packages die für den versuch verwendet werden

import numpy as np #numpy ist ein package mit allen mathematischen ausdrücken/
    ↳funktionen/operationen/etc.
from uncertainties import ufloat #Uncertainties ist für das automatische
    ↳berechnen von unsicherheiten
from uncertainties.umath import * #Importieren von rechnen mit uncertainties
    ↳(addition/multiplikation/etc.)
from uncertainties import unumpy as unp #kann arrays von unsicherheiten
    ↳verstehen (also eine liste, statt jede zahl einzeln einzutippen)
from IPython.display import Latex, HTML, Math, display # Um ergebnisse schöner
    ↳anzuzeigen. display() "ersetzt" quasi print(), und lässt latex fromatierung
    ↳zu
import matplotlib.pyplot as plt #Damit kann man graphen plotten
from scipy.optimize import curve_fit
from scipy.stats import linregress

import pandas as pd #Um die Spektroskopie daten zu lesen
from scipy.signal import find_peaks #um die maxima von listen (also auch
    ↳y-achse) zu finden
from scipy import constants as const
from scipy.odr import ODR, Model, RealData
```

1 Wärme und Wärmetransport

2 Wärmestrahlung

In diesem Experiment wird die Strahlungstemperatur einer Glühlampe bei zwei Betriebsbedingungen bestimmt, basierend auf der Abhängigkeit der Strahlungsintensität von der Temperatur.

2.0.1 Durchführung und Versuchsaufbau

Eine Glühlampe wird bei zwei unterschiedlichen Spannungen betrieben (6 V und 4 V). Das von ihr ausgesandte, Licht trifft auf eine Photozelle, deren Ausgangsspannung proportional zur Beleuchtungsstärke ist. Der Abstand r zwischen Glühlampe und Photodiode wird variiert. Für beide Betriebsspannungen wird die Spannung $U(r)$ der Photozelle für equidistante Werte von r gemessen.

Aus der Steigung der Ausgleichsgeraden a_x für beide Betriebspunkte werden die Verhältnisse der Lichtstärken I_x bestimmt:

$$\frac{I_1}{I_2} = \frac{a_1}{a_2}.$$

Die elektrischen Leistungen beider Betriebszustände werden dann über $P = UI$ berechnet, wobei Stromstärke und Spannung an der Lampe abgelesen und dann in das Stefan–Boltzmann-Gesetz eingesetzt wird:

$$\frac{P_1}{P_2} = \frac{T_1^4}{T_2^4}.$$

Zur Berechnung der Strahlungstemperaturen wird schließlich

$$\ln\left(\frac{I_1}{I_2}\right) = \left(\frac{1}{T_2} - \frac{1}{T_1}\right) \frac{ch}{k\lambda}$$

verwendet.

2.0.2 Verwendete Geräte und Unsicherheiten

Spannungsquelle TTi EL-155R: - Spannung: Auflösung 10 mV, Unsicherheit (0.3 % + 3 Digit)
 - Stromstärke: Auflösung 1 mA, Unsicherheit (0.5 % + 3 Digit)

Voltmeter UNI-T UT61B (bereich 40 mV):

- Auflösung 0.01 mV
 - Unsicherheit (0.8 % + 3 Digit)

2.0.3 Systematische Unsicherheiten

Ungenau Positionierung der Lampe und des Photodetektors führt zu Fehlern in den Abstandangaben.

2.1 Ergebnisse:

[4]: `w = 580*1e-9 #Licht wellenlänge nm`

`# 4V Lampe`

`U_4 = ufloat(4.05, 4*0.003 +0.03)`

`#V`

`I_4 = ufloat(3.53, 3.7*0.005+0.01)`

`#A`

`↪ aufgrund fluktuation nicht 0.003 digits`

`U4v = np.array([15.44, 14.85, 14.45, 14.07, 13.66, 13.22, 12.95, 12.57, 12.13, 11.82])`

`#mV mit Voltmeter gemessen`

`U4 = unp.uarray(U4v, 0.4)*1e-6`

```

P_1 = U_4*I_4

# 6V Lampe

U_6 = ufloat(6.06, 6*0.003 +0.03) #V
I_6 = ufloat(4.48, 4.65*0.005+0.01) #A

U6v = np.array([35.46, 33.38, 31.24, 29.24, 27.32, 25.33, 23.37, 22.00, 20.10,
↪18.10]) #mV mit Voltmeter gemessen
U6 = unp.uarray(U6v, 0.4)*1e-6

P_2 = U_6*I_6

# Längenmessung

u = 1
#lampe = ufloat(500, u)
#sensor = unp.uarray([1000, 980, 960, 935, 905, 870, 830, 785],u)
r = unp.uarray([0.5, 0.52, 0.542, 0.568, 0.598, 0.633, 0.675, 0.727, 0.794, 0.
↪882], u)*1e3
#r = lampe - sensor

# Linearer fit mit gewichteten unsicherheiten

x = 1 / (r**2)

x_nom = unp.nominal_values(x)
x_err = unp.std_devs(x)

y4_nom = unp.nominal_values(U4)
y4_err = unp.std_devs(U4)

y6_nom = unp.nominal_values(U6)
y6_err = unp.std_devs(U6)

def linear(B, x):
    return B[0] * x + B[1]

model = Model(linear)

# Fit für 4V

```

```

data4 = RealData(x_nom, y4_nom, sx=x_err, sy=y4_err)
odr4 = ODR(data4, model, beta0=[1, 1])
out4 = odr4.run()

k_4, d_4 = out4.beta
uk_4, ud_4 = out4.sd_beta

a1 = ufloat(k_4, uk_4)

# Fit für 6V

data6 = RealData(x_nom, y6_nom, sx=x_err, sy=y6_err)
odr6 = ODR(data6, model, beta0=[1, 1])
out6 = odr6.run()

k_6, d_6 = out6.beta
uk_6, ud_6 = out6.sd_beta

a2 = ufloat(k_6, uk_6)

n="\n"

def chi2_from_odr(out, x, y, sy):
    """
    Compute  $\chi^2$  and  $\chi^2/\text{red}$  from an ODR fit.
    out: result from odr.run()
    x, y: data (nominal values)
    sy: errors in y (standard deviations)
    """
    beta = out.beta # fitted [slope, intercept]
    y_fit = beta[0] * x + beta[1] # model
    chi2 = np.sum(((y - y_fit) / sy)**2)
    ndof = len(x) - len(beta)
    return chi2, chi2/ndof

# For 4 V fit
chi2_4, redchi2_4 = chi2_from_odr(out4, x_nom, y4_nom, y4_err)

# For 6 V fit
chi2_6, redchi2_6 = chi2_from_odr(out6, x_nom, y6_nom, y6_err)

plt.figure(figsize=(10, 6))

```

```

plt.errorbar(unc.nominal_values(1/r**2), unc.nominal_values(U6), yerr=unc.
    ↪std_devs(U6), fmt='o', capsize=1.5, label="Betriebsspannung 6V",
    ↪color="orange")
plt.errorbar(unc.nominal_values(1/r**2), unc.nominal_values(U4), yerr=unc.
    ↪std_devs(U4), fmt='o', capsize=1.5, label="Betriebsspannung 4V",
    ↪color="blue")
plt.plot(unc.nominal_values(1/r**2), k_6 * unc.nominal_values(1/r**2) + d_6,
    ↪'r-', label=fr"Linearer Fit 6V {n} $\chi^2_{\{4V\}} = \{chi2_4:.2f\}, \qquad
    ↪\chi^2/\mathrm{{dof}} = \{redchi2_4:.2f\}$")
plt.plot(unc.nominal_values(1/r**2), k_4 * unc.nominal_values(1/r**2) + d_4,
    ↪'g-', label=fr"Linearer Fit 4V {n} $\chi^2_{\{6V\}} = \{chi2_6:.2f\}, \qquad
    ↪\chi^2/\mathrm{{dof}} = \{redchi2_6:.2f\}$")
plt.xlabel("1/(Abstandsquadrat) [1/m²]")
plt.ylabel("Spannung U [V]")
plt.title("Beleuchtungsstärke in Zusammenhang mit Abstand")
plt.legend()
plt.grid(True)
plt.show()

display(Latex(fr"$a_{\{4V\}} = (\{a1:.2f\}) \mathrm{{mV \cdot mm^2}}$"))
display(Latex(fr"$a_{\{6V\}} = (\{a2:.2f\}) \mathrm{{mV \cdot mm^2}}$"))

I_ratio = a1/a2

P = P_1/P_2

faktor = (const.c * const.h)/(const.Boltzmann * w) #c*h/k*lambda

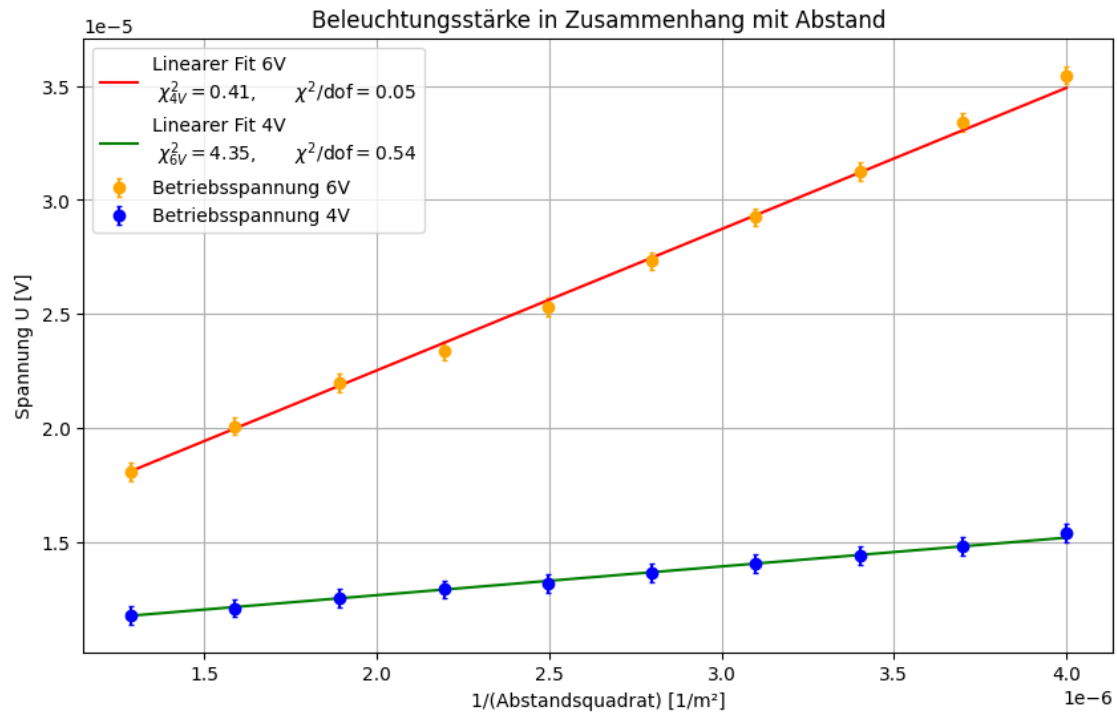
T_1 = (1/(np.log(unc.nominal_values(I_ratio)))) * ((P**(1/4))-1) * faktor
T_2 = (((T_1**4)*P_2)/P_1)**(1/4)

display(Latex(rf"$\frac{\{I_1\}}{\{I_2\}} = \{I_ratio:.3f\}$"))

display(Latex(f"$P_1 = (\{P_1:.2f\}) \text{ W}$"))
display(Latex(f"$P_2 = (\{P_2:.2f\}) \text{ W}$"))

display(Latex(f"$T_1 = (\{T_1:.2f\}) \text{ K}$"))
display(Latex(f"$T_2 = (\{T_2:.2f\}) \text{ K}$"))

```



$$a_{4V} = (1.26 + / - 0.02) \text{mV} \cdot \text{mm}^2$$

$$a_{6V} = (6.19 + / - 0.09) \text{mV} \cdot \text{mm}^2$$

$$\frac{I_1}{I_2} = 0.204 + / - 0.005$$

$$P_1 = (14.30 + / - 0.19) W$$

$$P_2 = (27.15 + / - 0.29) W$$

$$T_1 = (2308.73 + / - 56.58) K$$

$$T_2 = (2710.21 + / - 77.96) K$$

2.2 Diskussion

Beide Betriebseinstellungen passen sehr gut auf die Gefittete Funktion. Beide χ^2 werte sind nahe an 1 und weisen somit auf einen geeigneten Fit hin. Die Unsicherheiten der Steigungsgeraden und dem Daraus resultierenden Lichtstärke Verhältnis entspringen der Fit unsicherheit, wobei diese über ODS (Orthogonal Distance Regression) gewichtet wurde. Bei ausprobieren Höherer Werte werden jedoch die Unsicherheiten kaum verändert, weswegen dieser Fit wohl nicht vollständig oder gut gewichtet scheint.

2.3 Wärmeleitfähigkeit von Metallen - thermografische Bestimmung

In diesem Experiment wird die Wärmeleitfähigkeit eines Aluminiumstabes bestimmt, indem ein stationärer Wärmestrom erzeugt und der Temperaturgradient mithilfe einer Infrarotkamera gemessen wird.

2.3.1 Durchführung und Versuchsaufbau

Ein Aluminiumstab wird zwischen einer Wärmequelle (Heizwiderstand $R = 330 \Omega$) und einer Wärmesenke (massiver Kupferblock) befestigt.

Der Heizwiderstand wird mit einer konstanten Spannung von 14 V betrieben, wodurch sich eine konstante Heizleistung

$$P = \frac{U^2}{R}$$

ergibt.

Nach wenigen Minuten stellt sich ein stationärer Temperaturgradient entlang des Stabs ein welcher mit einer Infrarotkamera aufgenommen wird.

Daraufhin werden der **Emissionsgrad** ε des lackierten Aluminiumstabs sowie die **reflektierte Umgebungstemperatur (RTC)** mittels Referenzwerten im Gerät eingestellt.

Das Wärmebild wird im Programm BMT geöffnet. Entlang einer Linie wird das Temperaturprofil $T(x)$ extrahiert und als CSV-Datei gespeichert.

Die Pixelkoordinaten werden mittels gemessener Längen in meter umgerechnet. Entlang des Bereichs mit sichtlichen ++Temperaturverlauf wird ein linearer Fit durchgeführt:

$$T(x) = T_0 + \frac{\Delta T}{\Delta x} x.$$

Mit dem stationären Wärmeleitungsgesetz

$$\frac{\Delta Q}{\Delta t} = -\lambda A \frac{\Delta T}{\Delta x}$$

und dem bekannten Wärmestrom

$$\frac{\Delta Q}{\Delta t} = P$$

ergibt sich, nach messung der Querschnittsfläche A, die Wärmeleitfähigkeit:

$$\lambda = -\frac{P}{Ak}.$$

wobei k die Steigung der Gerade $\frac{\Delta T}{\Delta x}$

2.3.2 Verwendete Geräte und Unsicherheiten

Infrarotkamera (LIR-Bereich): * RTC-Korrektur

* Fokus und Bildschärfe

Schiebelehre:

* Auflösung typ. 0.05 mm

2.3.3 Systematische Unsicherheiten

Die Kamera misst nur Oberflächentemperaturen; Wärmeverluste durch Strahlung oder Konvektion können den linearen Temperaturgradienten leicht nicht-ideal machen.

2.4 Ergebnisse:

```
[3]: u = 0.1
l = ufloat(10, u) * (10**(-3))           #Länge mm
b = ufloat(4 + 3/20, u) * (10**(-3))     #Breite mm
Tl = ufloat(40.25, u) * (10**(-3))       #Temperaturlänge mm
T_l = unp.nominal_values(Tl)
A = l*b                                  #Querschnitt mm2

# 22° umgebung

#Anleitungsangaben

U = 14                                  #V
R = 330                                #Ohm
P = U**2/R                             #dQ/dt

df = pd.read_csv("Temperatur.csv", sep=',', decimal='.', engine='python')
px = df["d"].to_numpy()                 ↳
↳ #pixel
t = df["t"].to_numpy()                   ↳
↳ #temperatur
pxa = max(px)                           ↳ #px
↳ anzahl

slope, intercept, r, _, std = linregress((T_l/pxa)*px, t)           ↳
↳ #Länge pro Pixelanzahl mal pixel = länge in mm
steigung = ufloat(slope, std)
```



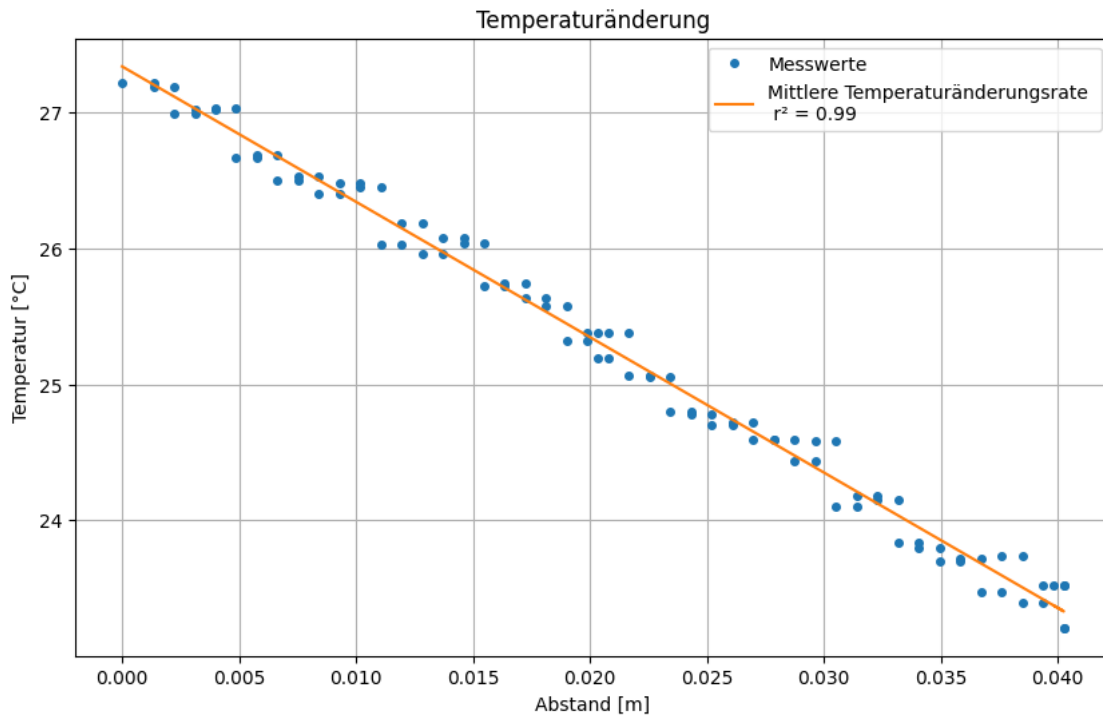
```

w = -(P/(steigung*A))
↳ Wärmeleitfähigkeit

#plot
plt.figure(figsize=(10, 6))
plt.plot((T_l/pxa)*px, t, 'o', markersize=4, label="Messwerte")
plt.plot((T_l/pxa)*px, slope*((T_l/pxa)*px)+intercept, '-', label=rf"Mittlere_
↳ Temperaturänderungsrate {n} r² = {r**2:.2f}")
plt.xlabel("Abstand [m]")
plt.ylabel("Temperatur [°C]")
plt.title("Temperaturänderung")
plt.legend()
plt.grid(True)
plt.show()

display(Latex(fr"$$\lambda = ({w:.2f}) \text{ J/msK}$$"))

```



$$\lambda = (143.57 \pm 4.03) \text{ J/msK}$$

2.5 Diskussion

Die Kalibrierung der Kamera war stark abweichend für geringe Werte, weshalb ein $\epsilon = 0.99$ angenommen wurde. Die Raumtemperatur wurde mit $(22 \pm 3)^\circ\text{C}$ gemessen, weshalb auch hier

die Kalibrierung der Kamera ungenau ist. Die Kamera hat eine Unsicherheit von 0.1°C . Wir erhalten aus den Messdaten einen Wert von $\lambda = (143.57 + / - 4.03) \frac{\text{J}}{\text{msK}}$ für die Wärmeleitfähigkeit. Das liegt nahe dem Wert von Duraluminium ($170 \frac{\text{J}}{\text{msK}}$). Die hohe Abweichung kommt wahrscheinlich von Systematischen Fehlern, der Wärmetransport durch Luft (Konvektion) und die Abnutzung der Legierung, als auch der fehlerbehafteten Kalibrierung der Kamera.